Rockchip Clock Configuration Detailed Explanation

ID: RK-KF-YF-030

Release Version: V1.1.1

Release Date: 2021-04-28

Security Level: □Top-Secret □Secret □Internal ■Public

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

Product Version

Chipset	Kernel Version
RK3399	4.4
RK3399	4.19

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Zhang Qing	2018-06-08	Initial version
V1.1.0	Zhang Qing	2019-11-11	Support for Linux 4.19
V1.1.1	Huang Ying	2021-04-28	Format modification

Contents

1. Clock Configuration

1.1 CRU Clock Configuration

1.1.1 CRU Clock Tree

The clock tree is too lengthy to describe here; for detailed information, refer to `cat /sys/kernel/debug/clk/clk_summary`.

1.1.2 Configuring Some Clocks to Remain Always On

During the debugging process, if you want to set certain clocks to remain always on, you can modify the `rk3399_cru_critical_clocks` structure by adding the clock names as needed:

```
drivers/clk/rockchip/clk-rk3399.c

    static const char *const rk3399_cru_critical_clocks[] __initconst = {
        "aclk_usb3_noc",
    };
```

The `clk` within this structure will default to calling the `clk_set_enable` interface when the system boots and the clk is initialized.

Note: If the clock is not always on and the driver device does not reference and enable this clock, `clk_disable_unused_subtree` (drivers/clk/clk.c) will be called to close the unused clocks after the clock initialization is completed. If you do not want the unused clocks to be closed, you can add the attribute `clk_ignore_unused` in the `dts`:

```
    chosen {
     bootargs = "earlyprintk=uart8250-32bit,0xff690000 clk_ignore_unused";
    };
```

1.1.3 CLK ID Acquisition

4.4 kernel dts references the clock based on the clk id, unlike 3.10 which indexes through clk name.
For detailed CLK ID acquisition, refer to section 2.3.2 of the document "Rockchip Clock Development Guide".

1.1.4 PLL Clock Configuration

For detailed introduction of Phase-Locked Loop (PLL), refer to sections 1.3 and 2.2.1 in the document "Rockchip Clock Development Guide". Generally, PLL should not be modified, especially when it is connected to display-related clocks, as re-setting PLL may cause flicker issues. PLL settings can be done in UBOOT or directly in the cru node.

1. Setting in dts

However, it is only called once during node initialization.

```
cru: clock-controller@ff760000 {
    assigned-clocks =
        <&cru ARMCLKL>, <&cru ARMCLKB>,
        <&cru PLL_NPLL>, <&cru PLL_CPLL>,
        <&cru PLL_GPLL>;
    assigned-clock-rates =
        <816000000>,   <816000000>,
        <600000000>, <500000000>,
        <800000000>;
};
```

ARMB PLL	ARML PLL	DDR PLL	GPLL	CPLL	VPLL	NPLL	USBPHYPLL	PMUPLL
1200	900	900	600	800	1200	1000	480	700

2. PLL Calculation Formula

Currently, there are two types of PLLs in the RK platform: one is NR\NF\NO (RK3066, RK3188, RK3288), and the other is REF\POSTDIV1\POSTDIV2 (RK3036, RK312X, RK322X, RK332X, RK336X, RK3399).

1. NR, NF, NO type, only integer division
   FREF = FIN / NR
   FOUTVCO = FREF * NF
   FOUTPOSTDIV = FOUTVCO / NO
   FREF range: 269MHZ - 2200MHZ, VCO range: 440MHz – 2200MHz, output frequency range: 27.5MHz – 2200MHz.
   NF_MAX: 4096, NR_MAX: 64, NO_MAX: 16 (only even numbers)
2. REF\POSTDIV1\POSTDIV2 type, supports both integer and fractional division
   Integer division:
   If DSMPD = 1 (DSM is disabled, "integer mode")
   FOUTVCO = FREF / REFDIV * FBDIV
   FOUTPOSTDIV = FOUTVCO / POSTDIV1 / POSTDIV2
   Fractional division:
   If DSMPD = 0 (DSM is enabled, "fractional mode")
   FOUTVCO = FREF / REFDIV * (FBDIV + FRAC / 2^24)
   FOUTPOSTDIV = FOUTVCO / POSTDIV1 / POSTDIV2
   VCO range: 440MHz – 3200MHz, output frequency range: 27.5MHz – 2200MHz.
   REFDIV_MAX: 63, FBDIV_MAX: 4095, POSTDIV1_MAX: 7, POSTDIV2_MAX: 7.

Note:
Generally, PLL does not require modifications, and the default configuration is sufficient. However, display clocks have jitter requirements, so there are special demands for PLLs that are exclusively used for display dclk. Some fine-tuning may be necessary.

PLL Setting Principles:
PLL generally requires frequencies to be as high as possible within the range of 600-1200M, making the VCO more suitable (jitter will be smaller). Current clock frequencies are mostly supported by automatic calculation, but automatic calculation can only ensure that VCO is within the aforementioned range and cannot guarantee that VCO is the best. Therefore, if there are strict requirements for VCO, the following approach can be taken: Set the PLL frequency higher and then divide it in the backend. For example: DCLK 50M, you can set CPLL to

1000M and then divide by 20. (This will definitely have better jitter than setting CPLL to 50M or 100M)
This modification method can be used in the cru node or device node with assigned-clock-rates (section 1), or in the driver, by first setting the PLL and then setting dclk.
You can add the required frequencies in the PLL frequency table and specify VCO.

Taking RK3399 4.4 kernel as an example:

```
static struct rockchip_pll_rate_table rk3399_vpll_rates[] = {
    /* _mhz, _refdiv, _fbdiv, _postdiv1, _postdiv2, _dsmpd, _frac */
    RK3036_PLL_RATE( 594000000, 1, 123, 5, 1, 0, 12582912),  /* vco = 2970000000 */
    RK3036_PLL_RATE( 593406593, 1, 123, 5, 1, 0, 10508804),  /* vco = 2967032965 */
    RK3036_PLL_RATE( 297000000, 1, 123, 5, 2, 0, 12582912),  /* vco = 2970000000 */
    RK3036_PLL_RATE( 296703297, 1, 123, 5, 2, 0, 10508807),  /* vco = 2967032970 */
    RK3036_PLL_RATE( 148500000, 1, 129, 7, 3, 0, 15728640),  /* vco = 3118500000 */
    RK3036_PLL_RATE( 148351648, 1, 123, 5, 4, 0, 10508800),  /* vco = 2967032960 */
    RK3036_PLL_RATE( 106500000, 1, 124, 7, 4, 0,  4194304),  /* vco = 2982000000 */
    RK3036_PLL_RATE(  74250000, 1, 129, 7, 6, 0, 15728640),  /* vco = 3118500000 */
    RK3036_PLL_RATE(  74175824, 1, 129, 7, 6, 0, 13550823),  /* vco = 3115384608 */
    RK3036_PLL_RATE(  65000000, 1, 113, 7, 6, 0, 12582912),  /* vco = 2730000000 */
    RK3036_PLL_RATE(  59340659, 1, 121, 7, 7, 0,  2581098),  /* vco = 2907692291 */
    RK3036_PLL_RATE(  54000000, 1, 110, 7, 7, 0,  4194304),  /* vco = 2646000000 */
    RK3036_PLL_RATE(  27000000, 1,  55, 7, 7, 0,  2097152),  /* vco = 1323000000 */
    RK3036_PLL_RATE(  26973027, 1,  55, 7, 7, 0,  1173232),  /* vco = 1321678323 */
    { /* sentinel */ },
};
```

You can modify the corresponding _refdiv, _fbdiv, _postdiv1, _postdiv2 in the table to achieve a suitable VCO. (For the processing method of 3.10 kernel, please refer to section 3.1 in the document "Rockchip Clock Development Guide")

1.1.5 CLK_TIMER Clock Configuration

Timer's clock is directly derived from 24M after gating bypass. Therefore, there is no concept of frequency setting, only the notion of clock on/off. If you want to configure the clk to be always on, please refer to section 1.1.2 of this document.
If the driver controls it by itself, the following operations can be performed:

```
    struct clk *clk;
    clk = clk_get(NULL, "clk_timer00");
    clk_prepare_enable(clk);
```

1.1.6 Bus Clock Configuration

Bus clocks are divided into high-speed and low-speed, with high-speed clocks including aclk_perihp, hclk_perihp, pclk_perihp, and low-speed clocks being aclk_perilp0, hclk_perilp0, pclk_perilp0, and hclk_perilp1, pclk_perilp1, which are configurable in terms of clock frequency. However, the sub-clocks under these clocks in the clock tree are gating, which means they can only be turned on and off, not set to a specific frequency. If

frequency modification is desired, it can only be done by altering the parent clock's frequency. The CCI serves as the inter-core communication bus clock ACLK_CCI, and the DDR bus clock is ACLK_CENTER.

Frequency Setting Method

Setting in dts (but only called once during node initialization)

```
cru: clock-controller@ff760000 {
    assigned-clocks =
        <&cru ACLK_PERIHP>, <&cru HCLK_PERIHP>,
        <&cru PCLK_PERIHP>,
        <&cru ACLK_PERILP0>, <&cru HCLK_PERILP0>,
        <&cru PCLK_PERILP0>,
        <&cru HCLK_PERILP1>, <&cru PCLK_PERILP1>,
        <&cru ACLK_CCI>, <&cru ACLK_CENTER>;
    assigned-clock-rates =
        <150000000>,   <75000000>,
        <37500000>,
        <100000000>,  <100000000>,
        <50000000>,
        <100000000>,   <50000000>,
        <40000000>, <40000000>;
};
```

Buses do not have separate drivers or nodes, so frequency settings are handled through the cru node using the assigned method.

Clock Frequency Range

In the bus, aclk is generally used for data transfer, while hclk and pclk are used for register read/write operations. The designed frequencies are as follows:

CLK	SIZEOFF	CLK	SIZEOFF
ACLK_PERIHP	300M	ACLK_PERILP0	300M
HCLK_PERIHP	150M	HCLK_PERILP0	150M
PCLK_PERIHP	150M	PCLK_PERILP0	150M
HCLK_PERILP1	150M	PCLK_PERILP1	150M
CCI	600M	ACLK_CENTER	300M

If overclocking is required, consider applying additional voltage (logic path voltage).

1.1.7 FCLK_CM0S Clock Configuration

The fclk_cm0s is located in the cru, while fclk_cm0s_src_pmu is in the pmucru, and there are differences in the clock settings between the two. For pmucru, please refer to section 1.2.4 in this chapter where fclk_cm0s is capable of configuring the clock, and all the clocks below it are gating (hclk_m0_perilp_noc\clk_m0_perilp_dec\dclk_m0_perilp\hclk_m0_perilp\sclk_m0_perilp), which can only be

turned on and off, not set to a specific frequency. If you wish to modify the frequency, you can only adjust the frequency of fclk_cm0s. Moreover, the frequency can only be derived from GPLL or CPLL.

Frequency Setting Method

Set in the dts, but it is only called once during node initialization.

```
    cru: clock-controller@ff760000 {
        assigned-clocks = <&cru FCLK_CM0S>;
        assigned-clock-rates = <100000000>;
    };
```

It can be placed in the cru node or within the device node.
Set the frequency in the driver
Buses generally do not have separate drivers, so it is usually set in the dts. If some drivers internally want to adjust this frequency, it is also possible.

```
    struct clk *clk;
    clk = clk_get(NULL, "fclk_cm0s");
    clk_set_rate(clk, rate);/* rate unit is hz */
```

Clock Frequency Range

The designed frequency for M0 is 100M. If overclocking is required, consider increasing the voltage (logic path voltage increase).

1.1.8 CLK_I2C Clock Configuration

It is important to note that I2C1, 2, 3, 5, 6, and 7 are configured in the cru module (I2C0, 4, and 8 are set in pmucru; refer to section 1.2.5 for frequency settings). For the 3399 chip, the I2C has two clocks: a control clock clk_i2c1 and a configuration clock pclk_i2c1. The frequency of the control clock can only be derived from the division of CPLL or GPLL, while the configuration clock frequency comes from pclk_perilp1, which only gates and cannot modify the frequency. If the frequency needs to be modified, then pclk_perilp1 must be changed (see 1.1.5).

Frequency Setting Method (Taking i2c1 as an Example)

1. Set in dts, but it is called only once during node initialization.

```
    i2c1: i2c@ff110000 {
        clocks = <&cru SCLK_I2C1>, <&cru PCLK_I2C1>;
        clock-names = "i2c", "pclk";
        assigned-clocks = <&cru SCLK_I2C1>;
        assigned-clock-rates = <50000000>;
    };
```

It can be placed in the cru node or within the device node.

2. Set frequency in the driver

```
In the I2C driver file drivers/i2c/busses/i2c-rk3x.c:
i2c->clk = devm_clk_get(&pdev->dev, "i2c");
i2c->pclk = devm_clk_get(&pdev->dev, "pclk");
clk_set_rate(i2c->clk, rate); /* rate unit is hz */
```

Clock Frequency Range

Generally, the control clock frequency should not exceed 100M, and the configuration clock should not exceed 100M. If overclocking is required, consider applying additional voltage (logic path voltage).

1.1.9 CLK_SPI Clock Configuration

It is important to note that spi0, 1, 2, 4, 5 are configured in the cru module (spi3's frequency is set in pmucru, refer to section 1.2.6), and the 3399 chip has two SPI clocks, one control clock clk_spi0 and one configuration clock pclk_spi0. The frequency of the control clock can only be derived from the division of CPLL or GPLL, while the configuration clock frequency for spi0, 1, 2, 4 is derived from pclk_perilp1, and spi5 from hclk_perilp1; they only perform gating and cannot modify the frequency. If the frequency needs to be changed, then pclk_perilp1 and hclk_perilp1 must be modified (see 1.1.5).

Frequency Setting Methods

1. Set in dts, but only called once during node initialization.

```
spi0: spi@ff1c0000 {
    clocks = <&cru SCLK_SPI0>, <&cru PCLK_SPI0>;
    clock-names = "spiclk", "apb_pclk";
    assigned-clocks = <&cru SCLK_SPI0>;
    assigned-clock-rates = <50000000>;
};
```

It can be placed in the cru node or within the device node.

2. Set frequency in the driver

In the SPI driver file drivers/spi/spi-rockchip.c:

```
rs->apb_pclk = devm_clk_get(&pdev->dev, "apb_pclk");
rs->spiclk = devm_clk_get(&pdev->dev, "spiclk");
clk_set_rate(rs->spiclk, rate); /* rate in Hz */
```

Clock Frequency Range

Generally, the control clock frequency should not exceed 50M, and the configuration clock should not exceed 50M. If overclocking is required, consider applying additional voltage (logic path voltage).

1.1.10 CLK_UART Clock Configuration

It should be noted that uart0, 1, 2, 3 are in the cru module (uart4 is set in pmucru, refer to section 1.2.8 for frequency setting), and the 3399 chip has two UART clocks, one control clock clk_uart0 and one configuration clock pclk_uart0. The control clock supports fractional and integer frequency division. The clk_uart0_div is obtained by integer division from CPLL, GPLL, USB480M, while clk_uart0_frac is derived from clk_uart0_div using a fractional divider (for fractional division, the input clock should be more than 20 times the output clock to avoid poor clock Jitter). Use integer division when possible, and resort to fractional division when integer division is not sufficient. The configuration clock frequency is derived from pclk_perilp1, which only gates and does not modify the frequency; if frequency modification is required, modify pclk_perilp1 (see 1.1.5).

Frequency Setting Methods

1. Set in dts, but only called once during node initialization.

```
uart0: serial@ff180000 {
    clocks = <&cru SCLK_UART0>, <&cru PCLK_UART0>;
    clock-names = "baudclk", "apb_pclk";
    assigned-clocks = <&cru SCLK_UART0>;
    assigned-clock-rates = <24000000>;
};
```

It can be placed in the cru node or within the device node.

2. Set frequency in the driver

UART driver file drivers/tty/serial/8250/8250_dw.c

```
data->clk = devm_clk_get(&pdev->dev, "baudclk");
data->pclk = devm_clk_get(&pdev->dev, "apb_pclk");
clk_set_rate(data->clk, rate);/* rate in hz */
```

Clock Frequency Range

This mainly depends on the baud rate required by the uart. Generally, the frequency of uart is the baud rate * 16 (HZ). Our platform typically supports 115200 and 1500000 by default, and other baud rates depend on whether the PLL frequency can be divided accordingly.

1.1.11 CLK_EMMC, CLK_SDIO, CLK_SDMMC Clock Configuration

These are somewhat unique due to internal dual-edge data capture, which requires a clock duty cycle of 50%, thus necessitating an even number of frequency divisions. EMMC has two clocks; clk_emmc is the controller clock, which requires even division. Aclk_emmc is the data transfer and configuration clock. SDIO has two clocks; clk_sdio is the controller clock, which requires even division. Hclk_sdio is the configuration clock. SDMMC also has two clocks; clk_sdmmc is the controller clock, which requires even division. Hclk_sdmmc is the configuration clock. The control clocks are configurable in frequency, and aclk_emmc, hclk_sdmmc can also be individually configured in frequency. Hclk_sdio is a gating clock derived from hclk_perilp1, and modifications to Hclk_sdio require changes to hclk_perilp1 (see 1.1.5).

Frequency Setting Methods

1. Set in dts, but only called once during node initialization.

```
    sdhci: sdhci@fe330000 {
        assigned-clocks = <&cru SCLK_EMMC>;
        assigned-clock-rates = <200000000>;
        clocks = <&cru SCLK_EMMC>, <&cru ACLK_EMMC>;
        clock-names = "clk_xin", "clk_ahb";
    };
    sdio0: dwmmc@fe310000 {
        assigned-clocks = <&cru SCLK_SDIO>;
        assigned-clock-rates = <100000000>;
        clocks = <&cru HCLK_SDIO>, <&cru SCLK_SDIO>,
            <&cru SCLK_SDIO_DRV>, <&cru SCLK_SDIO_SAMPLE>;
        clock-names = "biu", "ciu", "ciu-drive", "ciu-sample";
    };
    sdmmc: dwmmc@fe320000 {
        assigned-clocks = <&cru SCLK_SDMMC>;
        assigned-clock-rates = <100000000>;
        clocks = <&cru HCLK_SDMMC>, <&cru SCLK_SDMMC>,
            <&cru SCLK_SDMMC_DRV>, <&cru SCLK_SDMMC_SAMPLE>;
        clock-names = "biu", "ciu", "ciu-drive", "ciu-sample";
    };
```

It can be placed in the cru node or within the device node.

2. Set frequency in the driver

EMMC driver file drivers/mmc/host/Sdhci-of-arasan.c

```
    sdhci_arasan->clk_ahb = devm_clk_get(&pdev->dev, "clk_ahb");
    clk_xin = devm_clk_get(&pdev->dev, "clk_xin");
    clk_set_rate(clk_xin, rate);/* rate in hz */
```

Clock Frequency Range

The frequency range for the clocks is as follows; this is the input frequency, and there is an internal division by two within the module, so the actual frequency output by the module is half of the frequency seen on the clock tree.

CLK Name	Input Frequency
EMMC	200M
ACLK_EMMC	100M (300M/b 64Bit)
SDMMC/SDIO	300/240M (<=300M)

Note

For frequency settings, it should be noted that the parent of the control clocks for EMMC, SDIO, and SDMMC generally includes CPLL, GPLL, NPLL, PPLL, UPLL. Generally, among these PLLs, CPLL is exclusively occupied. If EMMC requires a 200M frequency, then the PLL frequency must be 400M/800M/1200M, so the

frequency that the control clock can divide into depends on the frequency of the PLL. It must be an even number of divisions to obtain the frequency (if the PLL only has 600M and 800M, then it can only divide into 150/200/300/400M, and the actual output frequency of the controller can only be 75, 100, 150, 200M).

1.1.12 Display-Related VOP, HDCP, EDP Clock Configuration with ISP

Display-related clock requirements are numerous, with dclk generally demanding any frequency due to varying display resolutions requiring different dclk frequencies. Meanwhile, aclk and Hclk, serving as data transfer and register configuration clocks, are typically fixed at a certain value and do not change. Modifying aclk and Hclk during display can cause display flickering and other issues.

DCLK

To support arbitrary resolutions, DCLK must be capable of outputting any frequency. In such cases, DCLK should exclusively occupy a PLL. For dual displays, two PLLs should independently supply DCLK (whether this is supported is usually determined during chip design, or it can be said that once dual display is supported, other special functions may not be necessary). On the RK3399 platform, defining the macro RK3399_TWO_PLL_FOR_VOP supports dual display with arbitrary frequencies, where dclk_vop0 exclusively occupies CPLL and dclk_vop1 exclusively occupies VPLL. Both dclk_vop0 and dclk_vop1 have the CLK_SET_RATE_PARENT attribute, meaning that whatever frequency dclk requires, the corresponding parent PLL will be set to that frequency. If the macro is not enabled, only dclk_vop0 exclusively occupies CPLL and can support arbitrary frequencies, while dclk_vop1 will be subject to the current parent's nearest frequency division.

```
#ifdef RK3399_TWO_PLL_FOR_VOP
    COMPOSITE(DCLK_VOP0_DIV, "dclk_vop0_div", mux_pll_src_vpll_cpll_gpll_p,
            CLK_SET_RATE_PARENT | CLK_SET_RATE_NO_REPARENT,
            RK3399_CLKSEL_CON(49), 8, 2, MFLAGS, 0, 8, DFLAGS,
            RK3399_CLKGATE_CON(10), 12, GFLAGS),
#else
    COMPOSITE(DCLK_VOP0_DIV, "dclk_vop0_div", mux_pll_src_vpll_cpll_gpll_p,
            CLK_SET_RATE_PARENT,
            RK3399_CLKSEL_CON(49), 8, 2, MFLAGS, 0, 8, DFLAGS,
            RK3399_CLKGATE_CON(10), 12, GFLAGS),
#endif

#ifdef RK3399_TWO_PLL_FOR_VOP
    COMPOSITE(DCLK_VOP1_DIV, "dclk_vop1_div", mux_pll_src_vpll_cpll_gpll_p,
            CLK_SET_RATE_PARENT | CLK_SET_RATE_NO_REPARENT,
            RK3399_CLKSEL_CON(50), 8, 2, MFLAGS, 0, 8, DFLAGS,
            RK3399_CLKGATE_CON(10), 13, GFLAGS),
#else
    COMPOSITE(DCLK_VOP1_DIV, "dclk_vop1_div", mux_pll_src_dmyvpll_cpll_gpll_p, 0,
            RK3399_CLKSEL_CON(50), 8, 2, MFLAGS, 0, 8, DFLAGS,
            RK3399_CLKGATE_CON(10), 13, GFLAGS),
#endif
```

ACLK, HCLK
Support for frequency settings is available, but if there is a display during Uboot, the frequency must remain unchanged when transitioning to the kernel; otherwise, screen flickering may occur. This means that the aclk and hclk frequencies in Uboot and the kernel must be consistent, as must their parents and parent frequencies.

Frequency Setting Methods

1. Setting in dts

Specify the parent of vop dclk, due to hardware design reasons of rk3399, the vop used for HDMI must be on vpll (same source as HDMI), and the parent of the other vop is cpll. Add to the respective dts in the display node.

```
    &vopb_rk_fb {
        assigned-clocks = <&cru DCLK_VOP0_DIV>;
        assigned-clock-parents = <&cru PLL_VPLL>;
    };
    &vopl_rk_fb {
        assigned-clocks = <&cru DCLK_VOP1_DIV>;
        assigned-clock-parents = <&cru PLL_CPLL>;
    };
```

ACLK and HCLK only need to be initialized once:

```
    &vopb {
        clocks = <&cru ACLK_VOP0>, <&cru DCLK_VOP0>, <&cru HCLK_VOP0>, <&cru
        DCLK_VOP0_DIV>;
        clock-names = "aclk_vop", "dclk_vop", "hclk_vop", "dclk_source";
        assigned-clocks = <&cru ACLK_VOP0>, <&cru HCLK_VOP0>;
        assigned-clock-rates = <400000000>, <100000000>;
    };
```

This can be placed in the cru node or within the device node. Similar treatment for HDCP, EDP, and ISP settings.

HDCP
ACLK_HDCP can be derived from CPLL, GPLL, PPLL frequency division, while HCLK_HDCP and PCLK_HDCP are derived from ACLK_HDCP frequency division. Each has an independent gating.

EDP
Only PCLK_EDP is available, which can be derived from CPLL, GPLL frequency division. Other EDP clocks are independent gatings under this.

ISP
ACLK_ISP0, ACLK_ISP1 for bus data transfer can be derived from CPLL, GPLL, PPLL frequency division, HCLK_ISP0, HCLK_ISP1 are register configuration clocks directly derived from ACLK frequency division, SCLK_ISP0, SCLK_ISP1 are controller clocks that can be derived from CPLL, GPLL, NPLL frequency division. When referencing clocks, ACLK references ACLK_ISP0_WRAPPER, and HCLK references HCLK_ISP0_WRAPPER, which will automatically turn on the parent clock to ensure the ACLK and HCLK clock paths are open.

2. Setting frequency in driver

vop driver file drivers/gpu/drm/rockchip/rockchip_drm_vop.c

```
    vop->hclk = devm_clk_get(vop->dev, "hclk_vop");
    if (IS_ERR(vop->hclk)) {
        dev_err(vop->dev, "failed to get hclk source\n");
        return PTR_ERR(vop->hclk);
    }
    vop->aclk = devm_clk_get(vop->dev, "aclk_vop");
    if (IS_ERR(vop->aclk)) {
        dev_err(vop->dev, "failed to get aclk source\n");
        return PTR_ERR(vop->aclk);
    }
    vop->dclk = devm_clk_get(vop->dev, "dclk_vop");
    if (IS_ERR(vop->dclk)) {
        dev_err(vop->dev, "failed to get dclk source\n");
        return PTR_ERR(vop->dclk);
    }
```

Clock Frequency Range

Dclk mainly depends on the screen resolution. Since PLL uses automatic calculation, jitter is not optimal. If adjustment is needed, the appropriate VCO can be calculated and filled into the PLL table (detailed VCO calculation method can be found in section 2.6.2 of the RK3399 datasheet).

```
Drivers/clk/rockchip/clk-rk3399.c

    static struct rockchip_pll_rate_table rk3399_vpll_rates[] = {
        /* _mhz, _refdiv, _fbdiv, _postdiv1, _postdiv2, _dsmpd, _frac */
        RK3036_PLL_RATE( 594000000, 1, 123, 5, 1, 0, 12582912),  /* vco = 2970000000 */
        RK3036_PLL_RATE( 593406593, 1, 123, 5, 1, 0, 10508804),  /* vco = 2967032965 */
        RK3036_PLL_RATE( 297000000, 1, 123, 5, 2, 0, 12582912),  /* vco = 2970000000 */
        RK3036_PLL_RATE( 296703297, 1, 123, 5, 2, 0, 10508807),  /* vco = 2967032970 */
        RK3036_PLL_RATE( 148500000, 1, 129, 7, 3, 0, 15728640),  /* vco = 3118500000 */
        RK3036_PLL_RATE( 148351648, 1, 123, 5, 4, 0, 10508800),  /* vco = 2967032960 */
        RK3036_PLL_RATE( 106500000, 1, 124, 7, 4, 0,  4194304),  /* vco = 2982000000 */
        RK3036_PLL_RATE(  74250000, 1, 129, 7, 6, 0, 15728640),  /* vco = 3118500000 */
        RK3036_PLL_RATE(  74175824, 1, 129, 7, 6, 0, 13550823),  /* vco = 3115384608 */
        RK3036_PLL_RATE(  65000000, 1, 113, 7, 6, 0, 12582912),  /* vco = 2730000000 */
        RK3036_PLL_RATE(  59340659, 1, 121, 7, 7, 0,  2581098),  /* vco = 2907692291 */
        RK3036_PLL_RATE(  54000000, 1, 110, 7, 7, 0,  4194304),  /* vco = 2646000000 */
        RK3036_PLL_RATE(  27000000, 1,  55, 7, 7, 0,  2097152),  /* vco = 1323000000 */
        RK3036_PLL_RATE(  26973027, 1,  55, 7, 7, 0,  1173232),  /* vco = 1321678323 */
        { /* sentinel */ },
    };
```

As for the corresponding frequency range (if 4K display bandwidth is insufficient, the frequency of ACLK can be increased accordingly, but attention should be paid to whether the voltage is sufficient and whether voltage boosting is required):

CLK	SIZEOFF	CLK	SIZEOFF
ACLK_VOP0/1	400M	ACLK_VIO	400M
DCLK_VOP0	600M	ACLK_ISP0/1	400M
DCLK_VOP1	300M	CLK_ISP0/1	500M
CLK_VOP0/1_PWM	200M	CLK_EDP	200M
ACLK_HDCP	400M		

1.1.13 Video Encoding and Decoding VDU, RGA, CODEC, IEP Clock Configuration

Primarily focuses on clock configurations related to VDU, RGA, CODEC, and IEP.

VDU
The ACLK_VDU can be derived from CPLL, GPLL, NPLL, or PPLL by frequency division, and HCLK_VDU is derived from ACLK. The controller clocks SCLK_VDU_CORE and SCLK_VDU_CA are derived from CPLL, GPLL, NPLL by frequency division.

RGA
The ACLK_RGA can be derived from CPLL, GPLL, NPLL, or PPLL by frequency division, and HCLK_RGA is derived from ACLK.

CODEC
The ACLK_VCODEC can be derived from CPLL, GPLL, NPLL, or PPLL by frequency division, and HCLK_VCODEC is derived from ACLK. The controller clock SCLK_RGA_CORE is derived from CPLL, GPLL, NPLL, or PPLL by frequency division.

IEP
The ACLK_IEP can be derived from CPLL, GPLL, NPLL, or PPLL by frequency division, and HCLK_IEP is derived from ACLK.

Frequency Setting Methods

1. Set in dts, but only called once during node initialization.

```
rkvdec: rkvdec@ff660000 {
    clocks = <&cru ACLK_VDU>, <&cru HCLK_VDU>,
    <&cru SCLK_VDU_CA>, <&cru SCLK_VDU_CORE>;
    clock-names = "aclk_vcodec", "hclk_vcodec",
    "clk_cabac", "clk_core";
    assigned-clocks = <&cru ACLK_VDU>, <&cru HCLK_VDU>,
    <&cru SCLK_VDU_CA>, <&cru SCLK_VDU_CORE>;
    assigned-clock-rates = <400000000>, <400000000>,
    <300000000>, <300000000>;
};
```

It can be placed in the cru node or within the device node (other modules handle similarly).

2. Set frequency in the driver

VCODEC's driver file drivers/video/rockchip/vcodec/vcodec_service.c

```
    pservice->aclk_vcodec = devm_clk_get(dev, "aclk_vcodec");
    pservice->hclk_vcodec = devm_clk_get(dev, "hclk_vcodec");
    pservice->clk_cabac = devm_clk_get(dev, "clk_cabac");
    pservice->clk_core = devm_clk_get(dev, "clk_core");
```

Clock Frequency Ranges

For the corresponding frequency ranges (if 4K video encoding and decoding bandwidth is insufficient, the frequency of ACLK can be increased accordingly, but attention must be paid to whether the voltage is sufficient and if voltage boosting is required):

CLK	SIZEOFF	CLK	SIZEOFF
ACLK_VCODEC	400M	ACLK_IEP	400M
ACLK_VDU	400M	ACLK_RGA	400M
CLK_VDU_CORE	300M	CLK_RGA_CORE	400M
CLK_VDU_CA	300M		

1.1.14 USB Clock Configuration

USB primarily includes aclk, Host, otg, and internal USB PHY.

USB3
ACLK_USB3 can be derived from CPLL, GPLL, or NPLL by frequency division.

HOST
HCLK_HOST0 and HCLK_HOST1 are both under HCLK_PERI, with only GATING properties. To modify the frequency, HCLK_PERI must be modified (for details, see section 1.1.5 of this document).

OTG
ACLK_USB3OTG0 and ACLK_USB3OTG1 are derived from ACLK_USB3, with only GATING properties. To modify the frequency, ACLK_USB3 must be modified. SCLK_USB3OTG0_REF is directly derived from 24M, with only GATING properties. SCLK_USB3OTG0_SUSPEND can be obtained from 24M or 32K and can be set to different frequencies.

PHY
SCLK_USBPHY1_480M is the output of the internal PLL phase-locked loop of USB PHY, which is generally a fixed frequency of 480M. The internal PLL output can directly select 24M output.

Frequency Setting Methods

1. Set in dts, but only called once during node initialization.

```
    usb_host0_ohci: usb@fe3a0000 {
        clocks = <&cru HCLK_HOST0>, <&cru HCLK_HOST0_ARB>,
             <&cru SCLK_USBPHY0_480M_SRC>;
        clock-names = "hclk_host0", "hclk_host0_arb", "usbphy0_480m";
```

```
    };

    usb_host1_ehci: usb@fe3c0000 {
        clocks = <&cru HCLK_HOST1>, <&cru HCLK_HOST1_ARB>,
             <&cru SCLK_USBPHY1_480M_SRC>;
        clock-names = "hclk_host1", "hclk_host1_arb", "usbphy1_480m";
    };
    usbdrd3_0: usb@fe800000 {
        clocks = <&cru SCLK_USB3OTG0_REF>, <&cru SCLK_USB3OTG0_SUSPEND>,
             <&cru ACLK_USB3OTG0>, <&cru ACLK_USB3_GRF>;
        clock-names = "ref_clk", "suspend_clk",
                  "bus_clk", "grf_clk";
    };
```

It can be placed in the cru node or within the device node (other modules are processed similarly).

2. Set frequency in the driver

VCODEC driver file drivers/usb/dwc3/dwc3-rockchip.c

```
    clk = of_clk_get(np, i);
    if (IS_ERR(clk)) {
        ret = PTR_ERR(clk);
        goto err0;
    }
    ret = clk_prepare_enable(clk);
    if (ret < 0) {
        clk_put(clk);
        goto err0;
    }
```

Clock Frequency Range

For the corresponding frequency range (if USB has large data copy, etc., the frequency of ACLK can be correspondingly increased, but attention should be paid to whether the voltage is sufficient and whether voltage boosting is needed), the SIZEOFF frequency of ACLK_USB is 400M.

1.1.15 CIF Clock Configuration

CIF primarily refers to SCLK_CIF_OUT, which may have clock requirements such as 24M or 27M. This clock source can be directly selected at 24M for frequency division or can choose CPLL, GPLL, NPLL and then divide the frequency.

Frequency Setting Method

1. Set in dts, but only called once during node initialization.

```
    cru: clock-controller@ff760000 {
        assigned-clocks = <&cru SCLK_CIF_OUT_SRC>, <&cru SCLK_CIF_OUT>;
        assigned-clock-rates = <800000000>, <27000000>;
    };
```

It can be placed in the cru node or within the device node (similar treatment for other modules).

1.2 PMUCRU Clock Configuration

1.2.1 PMUCRU Clock Tree

```
   pll_ppll                          1     1   676000000     0 0
      ppll                           3     3   676000000     0 0
         pclk_pmu_src                3     3    48285715     0 0
            pclk_wdt_m0_pmu          0     0    48285715     0 0
            pclk_uart4_pmu           0     0    48285715     0 0
            pclk_mailbox_pmu         0     0    48285715     0 0
            pclk_timer_pmu           0     0    48285715     0 0
            pclk_spi3_pmu            0     0    48285715     0 0
            pclk_rkpwm_pmu           1     1    48285715     0 0
            pclk_i2c8_pmu            0     0    48285715     0 0
            pclk_i2c4_pmu            0     0    48285715     0 0
            pclk_i2c0_pmu            0     0    48285715     0 0
            pclk_noc_pmu             1     1    48285715     0 0
            pclk_sgrf_pmu            0     0    48285715     0 0
            pclk_gpio1_pmu           0     0    48285715     0 0
            pclk_gpio0_pmu           0     0    48285715     0 0
            pclk_intmem1_pmu         0     0    48285715     0 0
            pclk_pmugrf_pmu          0     0    48285715     0 0
            pclk_pmu                 0     0    48285715     0 0
         clk_i2c8_pmu                0     0    48285715     0 0
         clk_i2c4_pmu                0     0    48285715     0 0
         clk_i2c0_pmu                0     0    48285715     0 0
         clk_wifi_div                0     0    26000000     0 0
            clk_wifi_pmu             0     0    26000000     0 0
            clk_wifi_frac            0     0     1300000     0 0
         clk_spi3_pmu                0     0    96571429     0 0
         fclk_cm0s_pmu_ppll_src      1     1   676000000     0 0
            fclk_cm0s_src_pmu        2     2    84500000     0 0
               hclk_noc_pmu          1     1    84500000     0 0
               dclk_cm0s_pmu         0     0    84500000     0 0
               hclk_cm0s_pmu         0     0    84500000     0 0
               sclk_cm0s_pmu         0     0    84500000     0 0
               fclk_cm0s_pmu         0     0    84500000     0 0
```

Note

The clock control mentioned above is all within the pmucru register.

1.2.2 Configuring Some Clocks to Always Be On

For the purpose of debugging, if you wish to set certain clocks to always be enabled,

you can modify the `rk3399_pmucru_critical_clocks` structure by adding the clock names as they exist.

```
Drivers/clk/rockchip/clk-rk3399.c

static const char *const rk3399_pmucru_critical_clocks[] __initconst = {
    "pclk_noc_pmu",
};
```

In this structure, the `clk` will call the `clk_set_enable` interface by default when the system boots and the `clk` is initialized.

1.2.3 PCLK_PMU Bus Clock Configuration

The bus clock is only configurable through `pclk_pmu_src`, and all subsequent clocks are gated (`pclk_wdt_m0_pmu`, `pclk_uart4_pmu`, `pclk_mailbox_pmu`, `pclk_timer_pmu`, `pclk_spi3_pmu`, `pclk_rkpwm_pmu`, `pclk_i2c8_pmu`, `pclk_i2c4_pmu`, `pclk_i2c0_pmu`, `pclk_noc_pmu`, `pclk_sgrf_pmu`, `pclk_gpio1_pmu`, `pclk_gpio0_pmu`, `pclk_intmem1_pmu`, `pclk_pmugrf_pmu`, `pclk_pmu`). They can only be turned on and off, not set to a specific frequency. If a frequency change is desired, it can only be done by altering the frequency of `pclk_pmu_src`. Moreover, the frequency can only be derived from the PPLL (divisible frequencies from 676M).

Frequency Setting Method

1. Set in the device tree source (dts), but it is called only once during node initialization.

```
    pmucru: pmu-clock-controller@ff750000 {
        assigned-clocks = <&pmucru PCLK_SRC_PMU>;
        assigned-clock-rates = <100000000>;
    };
```

It can be placed within the `pmucru` node or within the device node.

2. Set frequency within the driver

Buses generally do not have a separate driver, so frequency settings are typically done in the dts. However, if a driver wishes to adjust this frequency internally, it is also possible.

```
    struct clk *clk;
    clk = clk_get(NULL, "pclk_pmu_src");
    clk_set_rate(clk, rate); /* rate in Hz */
```

Clock Frequency Range

The IC design frequency is 50M. If overclocking is required, consider increasing voltage (logical path voltage increase).

1.2.4 PMU_M0 Clock Configuration

The bus clock can only be configured with PCLK_SRC_PMUfclk_cm0s_src_pmu, and all clocks below it are gating (hclk_noc_pmu\dclk_cm0s_pmu\hclk_cm0s_pmu\sclk_cm0s_pmu\fclk_cm0s_pmu), which can only be turned on or off, not set to a specific frequency. If you wish to modify the frequency, you can only change the frequency of fclk_cm0s_src_pmu. Moreover, the frequency can only be derived from PPLL or 24M.

Frequency Setting Method

1. Set in dts, but it is only called once during node initialization.

```
    pmucru: pmu-clock-controller@ff750000 {
        assigned-clocks = <&pmucru FCLK_CM0S_SRC_PMU>;
        assigned-clock-rates = <100000000>;
    };
```

It can be placed within the pmucru node or within the device node.

2. Set frequency in the driver

The bus generally does not have a separate driver, so it is usually set in dts. If some drivers internally want to adjust this frequency, it is also possible.

```
    struct clk *clk;
    clk = clk_get(NULL, "fclk_cm0s_src_pmu");
    clk_set_rate(clk, rate);/* rate unit is hz */
```

Clock Frequency Range

The IC design frequency is 100M. If overclocking is required, consider increasing voltage (logic path voltage increase).

1.2.5 PMU Bus Clock Configuration

The bus clock is only configurable for PCLK_SRC_PMU, with all subsequent clocks being gating (see clock tree for details), which can only be turned on and off, not set to specific frequencies. To change the frequency, one must modify the frequency of PCLK_SRC_PMU. Additionally, the frequency can only be derived from the PPLL.

Frequency Setting Method

1. Set in the dts file, but it is called only once during node initialization.

```
    pmucru: pmu-clock-controller@ff750000 {
        assigned-clocks = <&pmucru PCLK_SRC_PMU>;
        assigned-clock-rates = <100000000>;
    };
```

It can be placed either within the pmucru node or within the device node.

Clock Frequency Range

The IC design frequency is 100M. If overclocking is required, consider increasing the voltage (logic path voltage increase).

1.2.6 PMU_I2C Clock Configuration

It is important to note that I2c0\4\8 in the pmucru module, for the 3399 chip, the I2C has two clocks, one control clock clk_i2c0_pmu and one configuration clock pclk_i2c0_pmu. The control clock frequency can only be derived from PPLL (676M) by frequency division, while the configuration clock frequency is sourced from pclk_pmu_src, which only acts as a gating function and cannot modify the frequency. If the frequency needs to be modified, it is necessary to change pclk_pmu_src (see 1.2.1).

Frequency Setting Method (Taking i2c0 as an Example)

1. Set in the dts, but it is called only once during node initialization.

```
    i2c0: i2c@ff3c0000 {
        clocks =  <&pmucru SCLK_I2C0_PMU>, <&pmucru PCLK_I2C0_PMU>;
        clock-names = "i2c", "pclk";
        assigned-clocks = <&pmucru SCLK_I2C0_PMU>;
        assigned-clock-rates = <50000000>;
    }
```

It can be placed in the pmucru node or within the device node.

2. Set the frequency in the driver

In the I2C driver file drivers/i2c/busses/i2c-rk3x.c:

```
    i2c->clk = devm_clk_get(&pdev->dev, "i2c");
    i2c->pclk = devm_clk_get(&pdev->dev, "pclk");
    clk_set_rate(i2c->clk, rate);/* rate unit is hz */
```

Clock Frequency Range

Generally, the control clock frequency should not exceed 100M, and the configuration clock should not exceed 100M. If overclocking is required, consider applying additional voltage (logic path voltage).

1.2.7 PMU_SPI Clock Configuration

It is important to note that spi3 in the pmucru module of the 3399 chip has two clocks: one control clock clk_spi3_pmu and one configuration clock pclk_spi3_pmu. The control clock frequency can only be derived from the PPLL (676M) by frequency division, while the configuration clock frequency is sourced from pclk_pmu_src, which only gates and does not modify the frequency. If the frequency needs to be changed, it is necessary to modify pclk_pmu_src (see 1.2.1).

Frequency Setting Methods

1. Set in the dts, but only called once during node initialization.

```
    spi3: spi@ff350000 {
        clocks = <&pmucru SCLK_SPI3_PMU>, <&pmucru PCLK_SPI3_PMU>;
        clock-names = "spiclk", "apb_pclk";
        assigned-clocks = <&pmucru SCLK_SPI3_PMU>;
        assigned-clock-rates = <50000000>;
    };
```

It can be placed either in the pmucru node or within the device node.

2. Set frequency in the driver

In the SPI driver file drivers/spi/spi-rockchip.c:

```
    rs->apb_pclk = devm_clk_get(&pdev->dev, "apb_pclk");
    rs->spiclk = devm_clk_get(&pdev->dev, "spiclk");
    clk_set_rate(rs->spiclk, rate);/* rate unit is hz */
```

Clock Frequency Range

Generally, the control clock frequency should not exceed 50M, and the configuration clock should not exceed 50M. If overclocking is required, consider applying additional voltage (logic path voltage).

1.2.8 PMU_WIFI Clock Configuration

It is important to note that the WIFI in the pmucru module, for the 3399 chip, supports both fractional and integer frequency division. The `clk_wifi_div` is obtained from the integer division of PPLL, while `clk_wifi_frac` is derived from `clk_wifi_div` using a fractional divider (for fractional division, the input clock should be at least 20 times the output clock, otherwise, the clock Jitter will be very poor). When using, if the integer division can meet the requirements, use integer division; if not, use fractional division.

Frequency Setting Methods

1. Set in dts, but only called once during node initialization.

```
    pmucru: pmu-clock-controller@ff750000 {
        compatible = "rockchip,rk3399-pmucru";
        assigned-clocks = <&pmucru SCLK_WIFI_PMU>;
        assigned-clock-rates = <26000000>;
    };
```

It can be placed in the pmucru node or within the device node.

2. Set frequency in the driver

```
    struct clk *clk;
    clk = clk_get(NULL, "clk_wifi_pmu");
    clk_set_rate(clk, rate);/* rate in hz */
```

Clock Frequency Range

This mainly depends on the crystal used by the wifi module, with common ones being 24M, 26M, 37.4M, and 40M.

1.2.9 PMU_UART4 Clock Configuration

It should be noted that uart4 in the pmucru module, for the 3399 chip, uarti has two clocks, one control clock clk_uart4_pmu and one configuration clock pclk_uart4_pmu. The control clock supports fractional and integer frequency division. clk_uart4_div is obtained by integer division of PPLL, while clk_uart4_frac is derived from clk_uart4_div using a fractional divider (for fractional division, the input clock must be at least 20 times the output clock to avoid poor clock Jitter). Use integer division when it meets the requirements, and fractional division when it does not. The configuration clock frequency comes from pclk_uart4_pmu, which only gates and cannot modify the frequency; if the frequency needs to be changed, modify pclk_pmu_src (see 1.2.1).

Frequency Setting Methods

1. Set in dts, but only called once during node initialization.

```
    uart4: serial@ff370000 {
        clocks = <&pmucru SCLK_UART4_PMU>, <&pmucru PCLK_UART4_PMU>;
        clock-names = "baudclk", "apb_pclk";
        assigned-clocks = <&pmucru SCLK_UART4_PMU>;
        assigned-clock-rates = <24000000>;
    };
```

It can be placed in the pmucru node or within the device node.

2. Set frequency in the driver

UART driver file drivers/tty/serial/8250/8250_dw.c

```
    data->clk = devm_clk_get(&pdev->dev, "baudclk");
    data->pclk = devm_clk_get(&pdev->dev, "apb_pclk");
    clk_set_rate(data->clk, rate);/* rate in hz */
```

Clock Frequency Range

This mainly depends on the baud rate required by the uart. Generally, the uart frequency is the baud rate * 16 (HZ). Our platform typically supports 115200 and 1500000 by default, and other baud rates need to be checked specifically to see if the PLL frequency can be divided accordingly.

2. Clock Interdependencies

2.1 Normal Parent-Child Relationship

The clock structure diagram and clock tree are as follows:

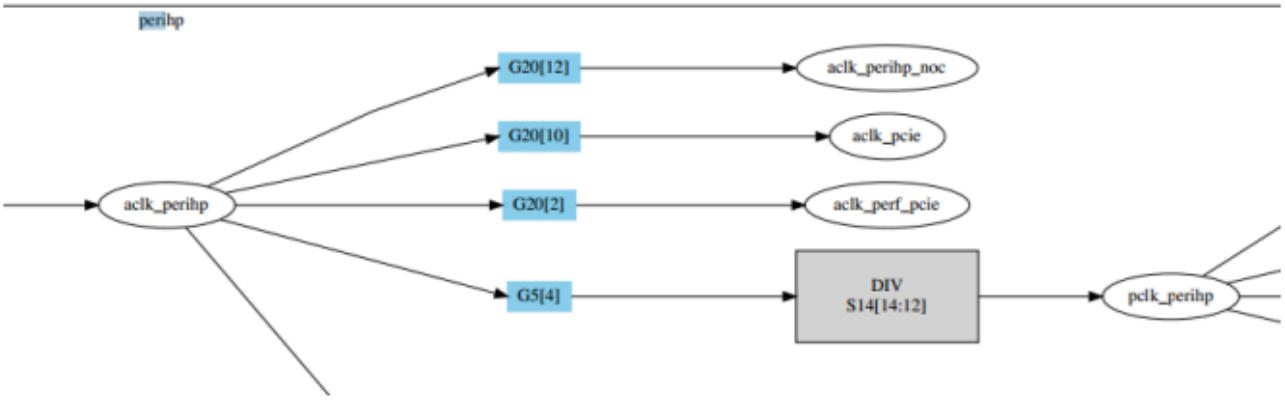

图 2-1 部分时钟结构图

时钟树的结构就是：

```
   clock                         enable_cnt  prepare_cnt        rate   accuracy   phase
----------------------------------------------------------------------------------------
      aclk_perihp                         4            5   133333334          0 0
         aclk_perihp_noc                  1            1   133333334          0 0
         aclk_pcie                        0            0   133333334          0 0
         aclk_perf_pcie                   0            0   133333334          0 0
         pclk_perihp                      3            3    33333334          0 0
            pclk_hsicphy                  0            0    33333334          0 0
            pclk_perihp_noc               1            1    33333334          0 0
            pclk_pcie                     0            0    33333334          0 0
            pclk_perihp_grf               1            1    33333334          0 0
```

The dependency relationship in a normal parent-child relationship is that when a child clock is enabled, the parent clock must also be enabled. The clock structure ensures this operation; simply enabling the child clock is sufficient. The clock structure will automatically index its parent clock and enable it. As long as its child clock is working, the parent clock cannot be disabled. Under normal circumstances, the enabling and disabling of clocks are managed by reference counting, such as the `enable_cnt` shown in the above diagram. When a child clock or the clock itself is enabled, the count is incremented, and when disabled, the count is decremented, until the count reaches zero, at which point the clock is disabled.

2.2 Inter-module NOC Reuse

When designing the NOC, some modules share the NOC, which requires that the NOC clock be enabled whenever any module is in use, and the entire clock path of the parent clock of the NOC must also be enabled.
The special requirements for the following clocks (which are already handled in the code to ensure the NOC clock is always on) are as follows:

表 2-1 NOC 依赖关系表 1

cci_noc	aclk_cci_noc0:G15[3]		
	aclk_cci_noc1:G15[4]		
	clk_dbg_noc:G15[5]		
peri_lp_noc	alive_noc（详细见下表格）		
	pmu_noc		
	emmc_noc		
	gmac_noc		
	usb3_noc		
	gic_noc		
	sdioaudio_noc		
	center_noc	msch0_noc	
		msch1_noc	
		iep_noc	
		rga_noc	
		perihp_noc	sd_noc
		vdu_noc	
		vcodec_noc	
		gpu_noc	
		edp_noc	
		vio_noc	isp0_noc
			isp1_noc
			hdcp_noc
			vopb_noc
			vopl_noc

表 2-2 NOC 依赖关系表 2

alive_noc	aclk_perilp0_noc:G25[7] hclk_perilp0_noc:G25[8] hclk_perilp1_noc:G25[9] pclk_perilp1_noc:G25[10]	alive_noc:PMUGRF0[6]	
		pclk_noc_pmu:PG1[6] hclk_noc_pmu:PG1[5]	
		aclk_emmc_noc:G32[9]	
		aclk_gmac_noc:G32[1] pclk_gmac_nocG32[3]	
		aclk_usb3_noc:G30[0]	
		aclk_gic_noc:G33[1]	
		sdioaudio_noc:G34[6]	
		aclk_center_main_noc:G19[0] aclk_center_peri_noc:G19[1] pclk_center_main_noc:G18[10]	msch0_noc:G18[4]
			msch1_noc:G18[9]
			aclk_iep_noc:G16[1] hclk_iep_noc:G16[3]
			aclk_rga_noc:G16[9] hclk_rga_noc:G16[11]
			aclk_perihp_noc:G20[12] hclk_perihp_noc:G20[13] pclk_perihp_noc:G20[14]
			aclk_vdu_noc:G17[9] hclk_vdu_noc:G17[11]
			aclk_vcodec_noc:G17[1] hclk_vcodec_noc:G17[3]
			gpu_noc
			pclk_edp_noc:G32[12]

2.3 GRF Reuse Between Different Modules

When designing GRF, some modules share GRF clocks, which requires that any module must enable the shared GRF clock when reading and writing GRF registers, and the entire clock path of the parent clock of GRF must be enabled.

The following are special requirements for clocks (currently handled in the code to ensure that GRF clocks are always on):

GRF	CON	GRF	CON
aclk_cci_grf	grf_a72_perf	pclk_grf(alive)	grf_iomux
aclk_cci_grf	grf_a53_perf	pclk_grf(alive)	grf_soc_con[0~8]
aclk_cci_grf	grf_cpu_status	pclk_grf(alive)	grf_usb3phy
aclk_cci_grf	grf_cpu_con	pclk_grf(alive)	grf_ddrc
		pclk_grf(alive)	grf_gpio2/3/4
pclk_perihp_grf	grf_hsic	pclk_grf(alive)	grf_io_vsel
pclk_perihp_grf	grf_hsicphy	pclk_grf(alive)	grf_saradc
pclk_perihp_grf	grf_usbhost0	pclk_grf(alive)	grf_tsadc
pclk_perihp_grf	grf_usbhost1	pclk_grf(alive)	grf_usb20_phy
pclk_perihp_grf	grf_usbphy	pclk_grf(alive)	grf_dll
pclk_perihp_grf	grf_usb20_host	pclk_grf(alive)	grf_alive_lf_ena
pclk_perihp_grf	grf_pcie	pclk_grf(alive)	grf_cphy
		pclk_grf(alive)	grf_uphy
pclk_vio_grf	grf_soc_con[9, 20~26]	pclk_grf(alive)	grf_pcie
pclk_vio_grf	grf_hdcp	pclk_grf(alive)	grf_sd
aclk_usb3_grf	grf_sta_usb3otg	aclk_gpu_grf	grf_gpu_perf
aclk_usb3_grf	grf_usb3_perf		

3. Clock Frequency Values

3.1 Configurable Clock Frequencies

Clock Name	Maximum Frequency	Configurable Frequencies
CCI	600M	(cpll/gpll/npll/vpll) / (1~32)
ACLK_CENTER	300M	(cpll/gpll/npll) / (1~32)
DDR_PCLK	200M	(cpll/gpll) / (1~32)
ACLK_PERIHP	300M	(cpll/gpll) / (1~32)

Clock Name	Maximum Frequency	Configurable Frequencies
ACLK_PERILP0	300M	(cpll/gpll) / (1~32)
AHB_PERILP1	150M	(cpll/gpll) / (1~32)
ACLK_VCODEC	400M	(cpll/gpll/npll/ppll) / (1~32)
ACLK_VDU	400M	(cpll/gpll/npll/ppll) / (1~32)
CLK_VDU_CORE	300M	(cpll/gpll/npll) / (1~32)
CLK_VDU_CA	300M	(cpll/gpll/npll) / (1~32)
ACLK_IEP	400M	(cpll/gpll/npll/ppll) / (1~32)
ACLK_RGA	400M	(cpll/gpll/npll/ppll) / (1~32)
CLK_RGA_CORE	400M	(cpll/gpll/npll/ppll) / (1~32)
EMMC	200M	(cpll/gpll/npll/ppll/upll/24M) / (1~128)
SDMMC/SDIO	400/300/240M	(cpll/gpll/npll/ppll/upll/24M) / (1~128)
CLK_PCIE_REF	24/100M	(npll/24M) / (1~128)
CLK_PCIE_CORE	250M	(cpll/gpll/npll) / (1~128)
CLK_PCIE_PM	25M/24M	(cpll/gpll/npll/24M) / (1~128)
ACLK_GMAC	300M	(cpll/gpll) / (1~32)
ACLK_EMMC	300M	(cpll/gpll) / (1~32)
M0-PERILP	300M	(cpll/gpll) / (1~32)
CRYPTO	200M	(cpll/gpll/ppll) / (1~32)
SPI	100M	(cpll/gpll) / (1~128)
I2S/SPDIF	Frac<=50M	(cpll/gpll) / (1~128)
SARADC	20M	24M/ (1~256)
TSADC	10M	24M/ (1~1024)
ACLK_VIO	400M	(cpll/gpll/ppll) / (1~32)
CLK_EDP	200M	(cpll/gpll) / (1~32)
ACLK_ISP0/1	400M	(cpll/gpll/ppll) / (1~32)
CLK_ISP0/1_JPE	500M	(cpll/gpll/npll) / (1~32)
ACLK_HDCP	400M	(cpll/gpll/ppll) / (1~32)
CLK_DP_CORE	200/100/10M	(cpll/gpll/npll) / (1~32)

Clock Name	Maximum Frequency	Configurable Frequencies
ACLK_VOP0/1	400M	(cpll/gpll/npll/vpll) / (1~32)
DCLK_VOP0	600M	(cpll/gpll/vpll) / (1~256)
DCLK_VOP1	300M	(cpll/gpll/vpll) / (1~256)
CLK_VOP0/1_PWM	200M	(cpll/gpll/vpll/24M) / (1~32)
ACLK_USB3	300M	(cpll/gpll/npll) / (1~32)
PCLK_ALIVE	100M	(gpll) / (1~32)
PCLK_PMU	100M	(ppll) / (1~32)
GPU	500M	(cpll/gpll/npll/upll/ppll) / (1~32)
M0-EC	200M	(cpll/gpll) / (1~32)